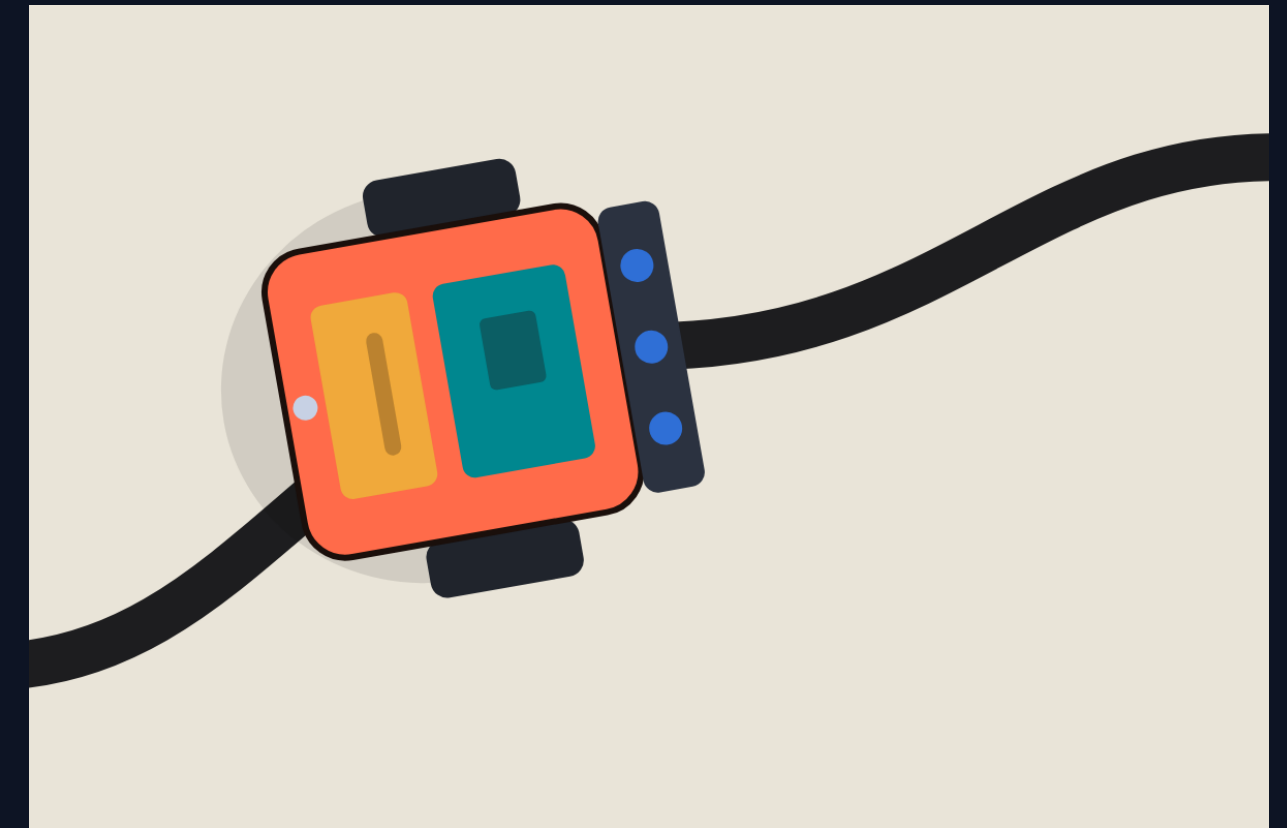


The robot worked yesterday

Today it loses the line. The deadline is approaching.

Arif Solmaz
İSTÜN Mechatronics Engineering



Elif + Deniz · One robot · One shared decision

Which file can we trust?

Elif and Deniz each sent a different “final version”.

THE PROBLEM

Which one is the latest?

23:47

Robot project


robot.ino


history: every change, who, when

Robot team

Kaan
Elif, can you send the latest code?

Elif
Sent robot_final.ino


Deniz
But I fixed the sensor in robot_v2...

Kaan
The robot left the line again. Which one is the latest??

Elif
No idea. Should we try them one by one?

On GitHub there is one `robot.ino` ; its old versions live in its history.

Calling a file “FINAL” does not record its history.

We have files. Where is the decision?

The team needs the change's history and reasoning.

What we have

Code files, several copies, and two people's memories.

What we need to find

What changed? Why? Which version should we continue from?

First goal: compare the records, not people's memories.

01 · FOLLOW THE ROBOT'S TRAIL

How do we compare yesterday and today?

There is a clue in the history: threshold 500 → 900.

2 BENEFIT

A time machine for your project

E Elif's screen

github.com/elif/my-robot/commit/7f3e2a1

elif / my-robot Public

<> Code Issues 0 Pull requests 0 Actions

Change sensor threshold

deniz committed yesterday 7f3e2a1

1 file changed

robot.ino +1 -1

```
3   int TURN_SPEED = 120; // speed in turns
4 - int THRESHOLD = 500; // sensor threshold
4 + int THRESHOLD = 900; // sensor threshold
5
6   void setup() {
```

<> Browse files

FOR YOU

A time machine for your project

✓ No change is ever lost.

✓ Who changed what, and when: all recorded.

✓ Don't be afraid to break things: every save is a way back.

Commit = a save point with a short note.

Found it: the threshold went from 500 to 900. You can look at the project as it was that day.

We found the change. We have not yet established the right threshold.

Which part of this is Git?

History lives on the computer. GitHub makes collaboration visible.

Git • local history

Records changes and shows how we arrived at a version.

GitHub • collaboration

Hosts the repository and brings tasks, proposals, and review together.

Git also works with Git servers other than GitHub.

01 · FOLLOW THE ROBOT'S TRAIL

What should we keep together?

The robot's code alone does not explain the project.

1 MINUTE

Put your project on GitHub

E Elif's screen

github.com/elif/my-robot

elif / my-robot Public

<> Code Issues 0 Pull requests 0 Actions

main Add file <> Code

E elif Add project files 2 Commits

photos Add project files

README.md Initial commit

circuit.png Add project files

project-report.pdf Add project files

robot.ino Add project files

README

my-robot

Line-following robot — class project

FOR YOU

Not just code: everything about your project

✓ Arduino code

✓ Circuit diagram

✓ Project report

✓ Photos

Repository = your project's folder on the internet.

Done: code, circuit diagram, report and photos are now online.

Repository = project files + recorded history

What changed on that line?

We are seeing the old and new values of one variable.

```
robot.ino
```

```
- const int ESIK = 500;  
+ const int ESIK = 900;
```

– previous line · + next line · commit: a recorded state

Is 900 wrong?

Git shows the change. We validate the threshold on the robot.

- History: what changed?
- Validation: how does it behave, under which conditions?
- Decision: what evidence would justify accepting it?

500 and 900 are example values in this fictional scenario.

01 • FOLLOW THE ROBOT'S TRAIL

Who is tracking the problem?

Issue: “Investigate the IR sensor threshold.”

3 BENEFIT

Your team's to-do list

E Elif's screen

FOR YOU

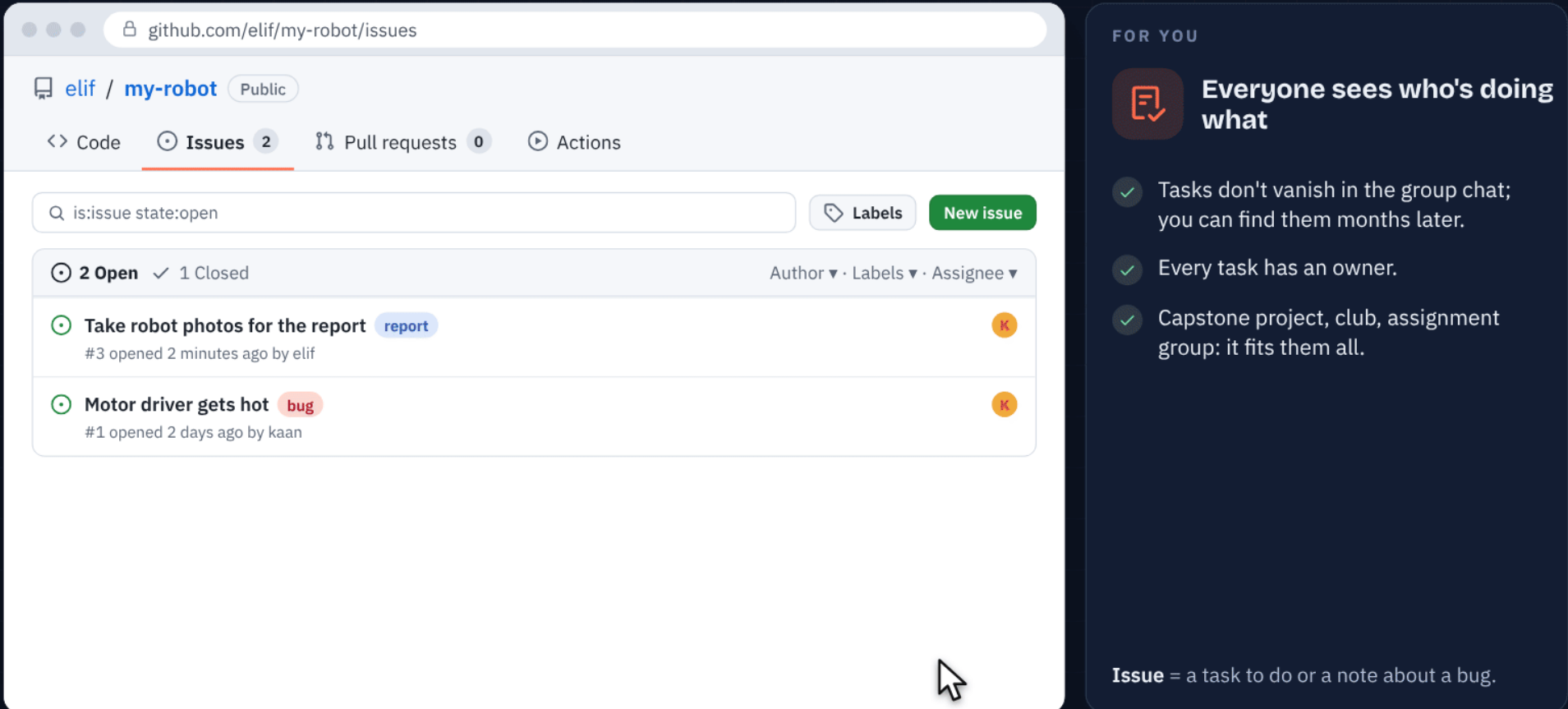
Everyone sees who's doing what

✓ Tasks don't vanish in the group chat; you can find them months later.

✓ Every task has an owner.

✓ Capstone project, club, assignment group: it fits them all.

Issue = a task to do or a note about a bug.



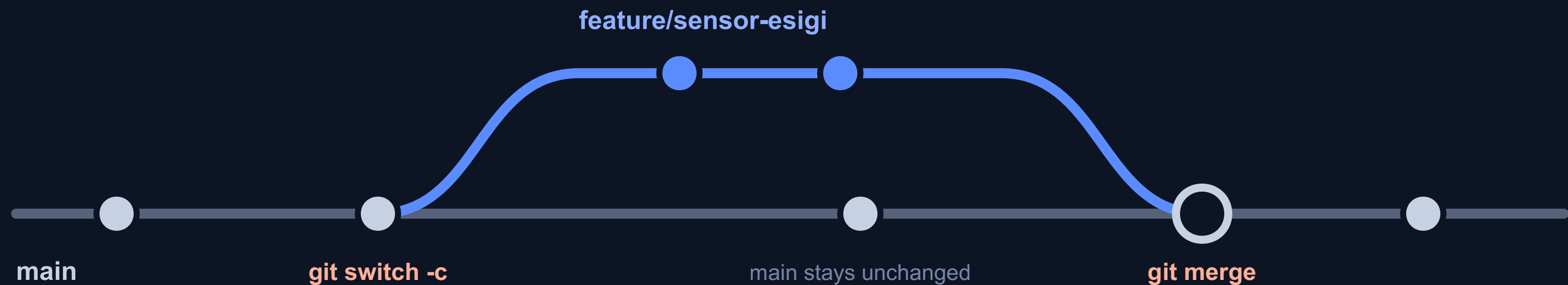
The screenshot shows a GitHub interface for a repository named 'my-robot'. At the top, there's a header with '3 BENEFIT' and 'Your team's to-do list'. Below this, the repository name 'elif / my-robot' is shown with a 'Public' label. Navigation tabs for 'Code', 'Issues' (selected), 'Pull requests', and 'Actions' are visible. A search bar contains 'is:issue state:open'. Below the search bar, there are filters for '2 Open' and '1 Closed'. The list of issues shows two open issues: 'Take robot photos for the report' (report) and 'Motor driver gets hot' (bug). To the right, a sidebar titled 'FOR YOU' explains the benefits of using issues, such as 'Everyone sees who's doing what' and 'Tasks don't vanish in the group chat; you can find them months later.' At the bottom of the sidebar, it states 'Issue = a task to do or a note about a bug.'

When it's done, **Close issue** : it leaves the list, the record stays.

GIF: the issue interface using a photo task. Our task is the sensor threshold.

How do we try it without changing main?

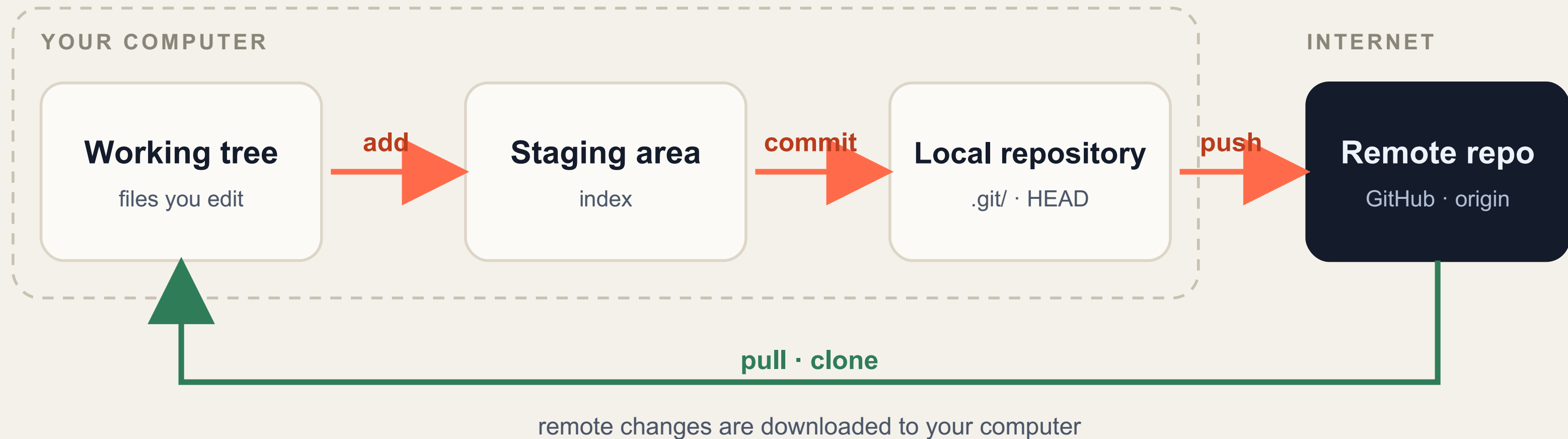
We will develop the candidate solution on a separate branch.



A branch organizes a trial. It does not prove the change is correct.

I saved it. Can Deniz see it?

Not yet. Editing, recording, and sharing are separate steps.



Edit → add → commit → push · Get the team's updates: pull

How do we get the same starting point?

Elif brings the project's files and history onto her computer.

```
git clone <repository-url>  
cd robot-projem
```

Clone brings history. A ZIP does not create a local Git history.

Which branch is the trial on?

Elif creates a separate line for the sensor task.

```
git switch -c feature/sensor-esigi  
git branch
```

Active branch: feature/sensor-esigi · This does not change main.

Which solution are we trying?

We prepare 900 → 500 as a candidate change.

```
robot.ino
```

```
- const int ESIK = 900;  
+ const int ESIK = 500;
```

Save → git status · History and GitHub have not changed yet.

What goes into this commit?

We select the sensor change. Other work can wait.

```
git add robot.ino  
git diff --cached
```

Add selects. There is no new commit or upload yet.

How do we record what we did?

A commit keeps the change and its purpose in history.

```
git commit -m "Candidate return to 500 for IR sensor threshold"  
git log --oneline
```

The record is local. “Candidate” means validation is still pending.

Can Deniz see it now?

After push, the candidate is visible on GitHub's task branch.

```
git push -u origin feature/sensor-esigi
```

A shared candidate ≠ an accepted change in main

Should we bring this into main?

A pull request proposes a change for the team to evaluate.

```
feature/sensor-esigi → main
```

Problem: The robot loses the line.

Candidate: ESIK, 900 → 500.

Reason: Investigate a return to the old value.

Validation: Measurement and behavior records pending.

Issue: what do we need? • PR: should we accept this proposal?

What will Deniz look for?

The changed line needs a clear reason and expected behavior.

- Why this value?
- Under which conditions will it be validated?
- Does it affect another behavior?

Feedback → new commit on the same branch → updated PR

What if two decisions change one line?

Elif's candidate is 500. Incoming main has 700.

```
robot.ino
```

```
<<<<<<< HEAD
```

```
const int ESIK = 500;
```

```
=====
```

```
const int ESIK = 700;
```

```
>>>>>>> main
```

We are on the task branch, merging main into it. Git cannot choose the right value.

Which one do we choose?

In this example, Elif and Deniz continue with the 500 candidate.

```
robot.ino  
  
const int ESIK = 500;
```

1. Evaluate the reasoning behind both changes together.
2. Remove the markers and save the chosen solution.
3. After validation, finish review through the same PR.

Removing conflict markers does not prove the robot's problem is fixed.

How does the decision enter shared history?

With sufficient review and validation, the proposal can enter main.

Before acceptance

A candidate on a task branch, open to discussion and changes.

After acceptance

A decision in shared main, with reasoning available in the project.

Make it visible → review it → accept it

GitHub is current. Is my computer?

We update local main separately.

```
git status  
git switch main  
git pull --ff-only
```

Push shares. Pull gets updates. A pull request proposes a merge.

Where does the next teammate start?

We leave running instructions alongside the code.

```
# Line-Following Robot

## Hardware
Arduino, IR sensors, motor driver.

## Getting started
Follow the wiring diagram.
Validate the threshold in your own conditions.
Record your observations and reasons for changes.
```

Purpose · hardware · wiring · running · known limits

Can everyone see it and change it?

Visibility and permission to modify are separate decisions.

Viewing

Public: anyone can view. Private: access is limited by permission.

Modifying

Writing to the main repository requires separate permission.

Passwords and access keys do not belong in project files.

Should we use the code we found?

An example needs compatible hardware and operating conditions.

- Does it match our hardware and environment?
- Are there explanations, examples, and known issues?
- What does the license allow?

Stars do not measure correctness on our robot.

Can we use the code we can see?

The license explains the conditions for sharing and use.

- Read the usage conditions.
- Keep required notices.
- State your own sharing conditions clearly.

Public visibility does not grant unrestricted usage rights.

How do we propose a fix to another project?

Push to our fork, then propose a PR to the original project.

1. Read the contribution guide.
2. Fork → clone → task branch → commit → push.
3. Propose a PR from your branch to the original project.

Fork: a remote copy. Clone: a copy on the computer.

04 • HAND OVER THE PROJECT

What does this project say about us?

Make the work behind the files understandable.

5 BENEFIT

Your GitHub profile is your portfolio

github.com/elif/my-robot

README.md

circuit.png

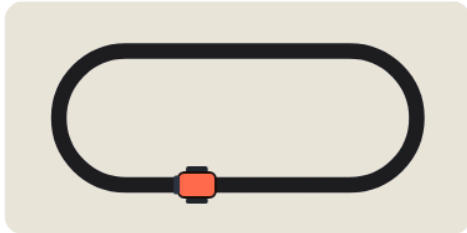
project-report.pdf

robot.ino

README

my-robot

Line-following robot — Mechatronics term project.



Parts

- Arduino Uno
- 2 DC motors
- 3 infrared sensors
- 7.4 V Li-ion battery

How it works

The sensors see the black line. When the robot drifts, it adjusts the motor speeds and gets back on track.

Elif Aydın

Mechatronics Engineering · Year 3

CV

PROJECTS

github.com/elif

FOR YOU



Your projects = your portfolio

✓

Teachers and employers see what you built, not just read about it.

✓

Every project is a showcase: photos, diagrams and notes in the README.

✓

Add the link to your CV for internships and jobs.

README = your project's front page: what, why, how.

Applying for an internship or a job? One line on your CV: github.com/elif

Problem · your contribution · reasoning · demonstration · known limits

Where can we continue after class?

Return to official resources when a need arises.

- Concepts: Pro Git
- Operation details: GitHub Docs
- Practice at your own pace: GitHub Skills

Setup and commands: 12 reference slides • Student tools: GitHub Education

What does the team have now?

We can explain which change we shared and why.

At the start

Different “final” files. Lost reasoning. Unclear local versions.

Now

Threshold reasoning in the PR. Current main on the team’s computers. A README ready for handover.

Git records the change. The team decides with a reason.

What if it loses the line tomorrow?

Our first question is clear: “What changed, and what is the evidence?”

Where is the change? Who can see it? Why accept it?

[Git and GitHub learning resources](#)

[Pro Git · English](#)

[GitHub Skills](#)

[GitHub Education](#)

Installing Git

Use the official installation route for your operating system.

```
git --version
```

Windows

Git for Windows, with Git Bash or a suitable terminal.

macOS and Linux

A package or installation method suited to the operating system.

Version numbers vary. Current instructions: git-scm.com/install

Commit identity

Configure the name and email that will appear in commits.

```
git config --global user.name "Elif Örnek"  
git config --global user.email "elif@example.com"  
git config --global init.defaultBranch main
```

The name and email are examples. These settings do not authenticate you to GitHub.

Editor and terminal

The same Git operation can be done in a terminal or a graphical interface.

Terminal

Commands and options are explicit, making steps easy to share.

VS Code

Files, diffs, Source Control, and an integrated terminal in one window.

Open the project with Open Folder; show the terminal with View → Terminal.

.git and .gitignore

The history database and tracking preferences are stored separately.

```
robot-projem/  
  .git/  
  .gitignore  
  README.md  
  robot.ino  
  
# Example .gitignore  
build/  
.env
```

Do not edit .git by hand. .gitignore does not remove tracked files from history.

An SSH key pair

Add the public key to GitHub; keep the private key to yourself.

```
ssh-keygen -t ed25519 -C "elif@example.com"
```

```
# File to add to GitHub:
```

```
~/.ssh/id_ed25519.pub
```

```
# Private key: do not share
```

```
~/.ssh/id_ed25519
```

A passphrase adds protection. Do not overwrite an existing key.

SSH agent and connection

Agent setup depends on the operating system.

```
# Bash / Git Bash example
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519

# Check the GitHub connection
ssh -T git@github.com
```

macOS Keychain and Windows OpenSSH steps may differ; follow the official instructions.

Turn a local folder into a repository

This example assumes an empty remote repository on GitHub.

```
cd robot-projem  
git init -b main  
git add README.md robot.ino  
git commit -m "Initial robot project commit"  
git remote add origin <repository-url>  
git remote -v
```

README.md and robot.ino already exist. If the remote has initial commits, clone may be simpler.

First push and tracking

-u records which remote branch the local branch tracks.

```
git push -u origin main
```

```
# After setting up tracking:
```

```
git push
```

```
git pull --ff-only
```

Check the branch name with `git branch`. Commands must use the project's actual branch name.

Branch and diff commands

Make the before and after sides of a comparison explicit.

```
git branch
git switch -c feature/sensor-esigi

# Compare branch tips: main → feature
git diff main feature/sensor-esigi -- robot.ino

# Feature changes from the common starting point:
git diff main...feature/sensor-esigi
```

Tab completion depends on the shell. The typed prefix must match the start of the branch name.

Choose what you want to undo

Files, the staging area, and history require different operations.

Unstage a change

```
git restore --staged README.md
```

The edit stays in the working file.

Reverse a shared change

```
git revert <commit>
```

Creates a new commit that reverses an earlier effect.

`git reset HEAD~1` moves the branch to the previous commit; the default mixed mode preserves working files.

Hard reset and recovery limits

A hard reset can discard uncommitted work.

```
# Only after checking the effects:  
git reset --hard <commit>  
  
# Inspect local reference movements:  
git reflog
```

Reflog may help find earlier commits; it cannot guarantee recovery of all unrecorded work.

Quick command reference

Commands are shorthand for the concepts in the main lecture.

```
git clone <repository-url>
# Edit and save the file
git status
git add <file>
git commit -m "What changed and why?"
git push

# Get the team's updates
git pull --ff-only
# Create a task branch
git switch -c feature/<task>
```

Replace placeholders with real values. Installation, identity, and permissions must be in place.